

# LEXSS: Bypassing Lexical Parsing Security Controls

**LEXSS: Bypassing Lexical Parsing Security Controls** - @bishopfox, Bishop Fox.

- Published: date not stated
- Original: <<https://bishopfox.com/blog/lexss-bypassing-lexical-parsing-security-controls>>
- Preserved from: <https://bishopfox.com/blog/lexss-bypassing-lexical-parsing-security-controls> (live) on 2026-08-10
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

## Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

**TL;DR** By using special HTML tags that leverage HTML parsing logic, it is possible to achieve cross-site scripting (XSS) even in instances where lexical parsers are used to nullify dangerous content. The primary goal in exploiting these types of XSS vulnerabilities is to get the sanitizing lexical parser to view the data as text data and not computer instructions (e.g., JavaScript instructions). This type of attack is possible when the HTML parser and the sanitizing lexical parsing do not parse the data in the same manner.

### Introduction to Key Concepts

Note: this blog post assumes some previous knowledge of XSS (better described as JavaScript injection) and a basic understanding of HTML. For a high-level primer, [head over to our XSS overview writeup](#).

### Cross-site Scripting (XSS) Protections

XSS protections come in many forms. In the early days of preventing XSS and occasionally still today, regular expressions (regex) were used to examine user input for "dangerous" strings. A simplified example is that if a user provided input containing `<script>`, the regex would match that exact string and remove it. However, this type of regex-based XSS protection often misses the mark, as there are many ways to formulate a JavaScript string of code to bypass the protection. In the same simplified example, simply capitalizing a letter in `<scRipt>` would bypass the filter and result in XSS. It is therefore not advisable to attempt a regex filter to prevent XSS.

So, let's talk about a better solution. Contextual output encoding is a form of XSS protection that transforms special characters such as `"<"` and `">"` into harmless HTML encoded output. This form of protection places the user's input within the source to ensure any JavaScript or HTML-usable characters that could result in JavaScript execution or HTML rendering are transferred into a non-

dangerous, HTML entity encoded form. This is a great way to handle user input that renders in the application; in fact, many contemporary front-end frameworks will now perform output encoding by default (e.g., ReactJS and Angular). However, this form of protection can limit an application's ability to allow users to include some types of HTML-enabled rich text.

What if users require the ability to include some HTML such as images, links, and rich text? This is where lexical parsing comes into play.

### Cross-site Scripting (XSS) Protections via Lexical Parsing

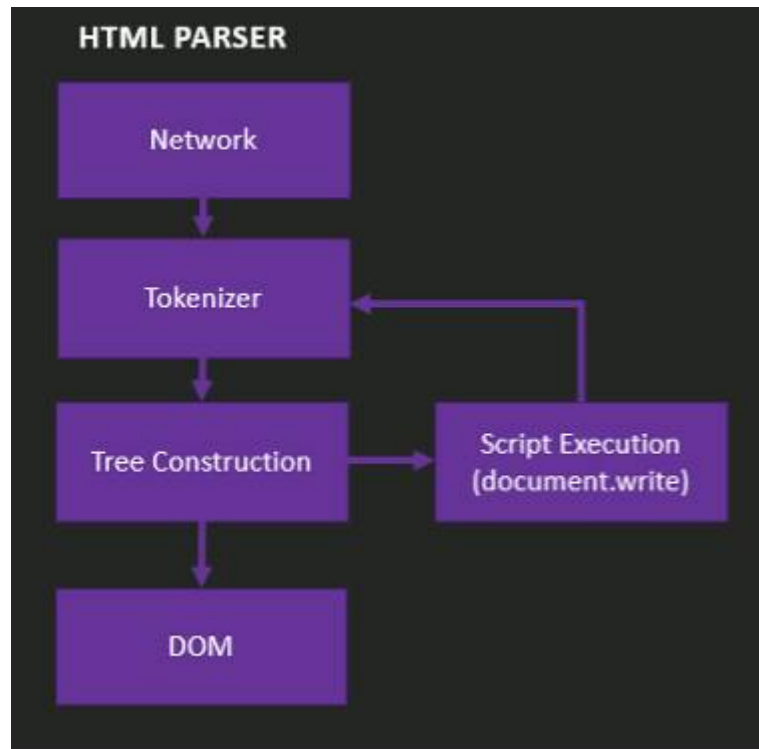
Lexical parsing is a very sophisticated way of preventing XSS because it evaluates whether the data is instructions or plaintext before performing additional logic such as blocking or encoding the data. At a high level, lexical parsing can be described as parsing that will separate user data (i.e., non-dangerous textual content) from computer instructions (i.e., JavaScript and certain dangerous HTML tags). In instances where the user is allowed a subset of HTML by design, this type of parsing can be used to determine what is allowed content and what will be blocked or sanitized.

Some examples that allow a subset of HTML by design are rich-text editors, email clients, What-You-See-Is-What-You-Get (WYSIWYG) HTML editors like TinyMCE or Froala, and sanitization libraries such as DOMPurify. Among these examples, this form of lexical parsing protection is commonplace.

However, lexical parsing as a form of XSS protection can be exploited when the HTML parser and sanitizing lexical parser do not process the data in the exact same manner. This blog post focuses on how in some cases it is possible to trick lexical parsers by leveraging the HTML parsing logic to inject JavaScript despite the sophisticated protections. In order to understand how to exploit these XSS issues, we must first examine how HTML is parsed; caveats and special cases during processing of data; and how the sanitizing parsers work.

### How the Data Flows Through the HTML Parser

To understand how we can achieve XSS in an application that uses lexical analysis on HTML input, we first must look at how HTML is parsed and how content is determined to be either data or instructions. The figure below is a visualization of the HTML parser's order of operations:



The steps in this visualization are as follows:

- **Network** – This stage refers to the transfer of input as bytes to the parser.
- **Tokenizer** – Tokenization is where the lexical parsing occurs. The parser will separate text data from computer instructions. To do this, the tokenizer will switch contexts between data states depending on the element it encounters and return the values as tokens. This is covered in more detail in the Context State section.
- **Tree Construction** – The tokens returned from the tokenization stage are placed in a tree structure; each of the tree branches is known as a node. For a clearer picture of what this looks like in practice, let's examine the following HTML snippet:

```
<!DOCTYPE html>
<body>
<div>
Hello World
<a href=https://bishopfox.com>Example</a></div>
</body>
```

The figure below shows what this looks like in the document object model (DOM) tree structure:



Our goal as an attacker is to control the node in this stage of HTML parsing. As my mentor Joe DeMesy once described it to me, if you can control a node's context and content, you will have XSS.

- **Script Execution** – This stage refers to when JavaScript alters the DOM. Additional details about this stage are out of the scope of this blog post.
- *\*DOM \**– The end state of the processing where the document object model is built.

**\*\*KEY TAKEAWAY: \*\***Now you should have a high-level understanding of how data flows through the HTML parsing process and how the information is organized, which will come into play during exploitation.

### The Concept of the HTML Parser's Context State

During the `tokenization` stage, the HTML parser will sort the HTML elements into different categories of data states known as the `Context State` . The [HTML specification](#) lists the `Context State` switching elements as follows:

Set the state of the HTML parser tokenization stage as follows, switching on the context element:

`title`

`textarea`

Switch the tokenizer to the RCDATA state.

`style`

`xmp`

`iframe`

`noembed`

`noframes`

Switch the tokenizer to the RAWTEXT state .

`script`

Switch the tokenizer to the script data state

`noscript`

If the scripting flag is enabled, switch the tokenizer to the RAWTEXT state. Otherwise, leave the tokenizer in the data state

## plaintext

Switch the tokenizer to the PLAINTEXT state.\*\* Any other element \*\*Leave the tokenizer in the data state.

The Context State refers to the context in which the HTML node will be parsed. What this means is that when the parser views any given <element>, it will determine the context of the node it is creating based on that element. This can take the form of text data (represented as either RCDATA, PLAINTEXT, or RAWTEXT states in the HTML parser) or computer instructions, which are represented as data state. A conceptual understanding of the Context State will be imperative for exploitation.

The visualization below shows what some of these context states look like in practice:

### RCDATA example:

Input: `<textarea><img src=x></textarea>`

Output: `<textarea><img src=x></textarea>`

Screenshot of output:

[\[image: screenshot of output\]](#)

DOM tree:

[\[image: DOM Tree 1\]](#)

### RAWTEXT example:

Input: `<xmp><img src=x></xmp>`

Output: `<xmp><img src=x></xmp>`

Screenshot of output:

[\[image: screenshot of outputstyle=\]](#)

DOM tree:

[\[image: DOM Tree 2\]](#)

Input: `<img src=x>`

Output: `<img src=x>`

Screenshot of output:

[\[image: Intentional broken image representation\]](#)

DOM tree:

[\[image: DOM tree 3\]](#)

Note that the `data state` is the only state that attempted to load an image. This is because data is a computer instruction and not simply text data.

### KEY TAKEAWAY:

Different supplied elements alter how data in those elements is parsed and rendered by switching the Context State of the data.

## Namespaces – Foreign Content and Leveraging the Unexpected Behavior

The browser's HTML parser understands more than just HTML; it can switch between three distinct namespaces: HTML, MathML, and SVG.

During HTML parsing, if either a `<svg>` or `<math>` namespace element (tag) is encountered, the parser will switch context to the respective namespace. What this context switch means to us is the parser is no longer parsing as HTML but rather MathML or SVG.

This namespace context switch results in unexpected behavior when HTML is embedded in MathML/SVG, as each namespace has its own distinct elements and parses slightly differently. As penetration testers, we can exploit this logic in some instances to confuse the parser into allowing XSS.

Michał Bentkowski's DOMPurify [writeup](#) provides a more in-depth look on namespace confusion, including cutting-edge research and a great example.

### KEY TAKEAWAY:

The HTML parser will context switch to separate namespaces when it encounters MathML or SVG elements, which can be used to confuse the parser.

### Sanitizing Lexical Parsing Flow

To exploit sanitizing lexical parsers, we need to understand the general flow of how they work. At a high level, the general flow is as follows:

- User-supplied data is parsed as HTML by the browser's HTML parser
- The data is parsed and sanitized by the lexical parser
- The data is parsed again by the browser's HTML parser

This flow is depicted below:

[\[image: flow of data parsing\]](#)

The goal of exploitation is to provide HTML that will trick the sanitizing parser into believing the provided input is non-dangerous text data (RCDATA, RAWTEXT, or PLAINTEXT) when it is actually computer instructions (data state). This is often possible for several reasons: HTML is not designed to be parsed twice; slight variations in parsing can occur between the initial HTML parser and the sanitizing parser; and sanitizing parsers often implement their own processing logic.

### Test Case 1 = TinyMCE XSS

TinyMCE is a What-You-See-Is-What-You-Get (WYSIWYG) HTML text editor and JavaScript library. It is typically included in third-party websites to provide text editing functionality, including HTML text.

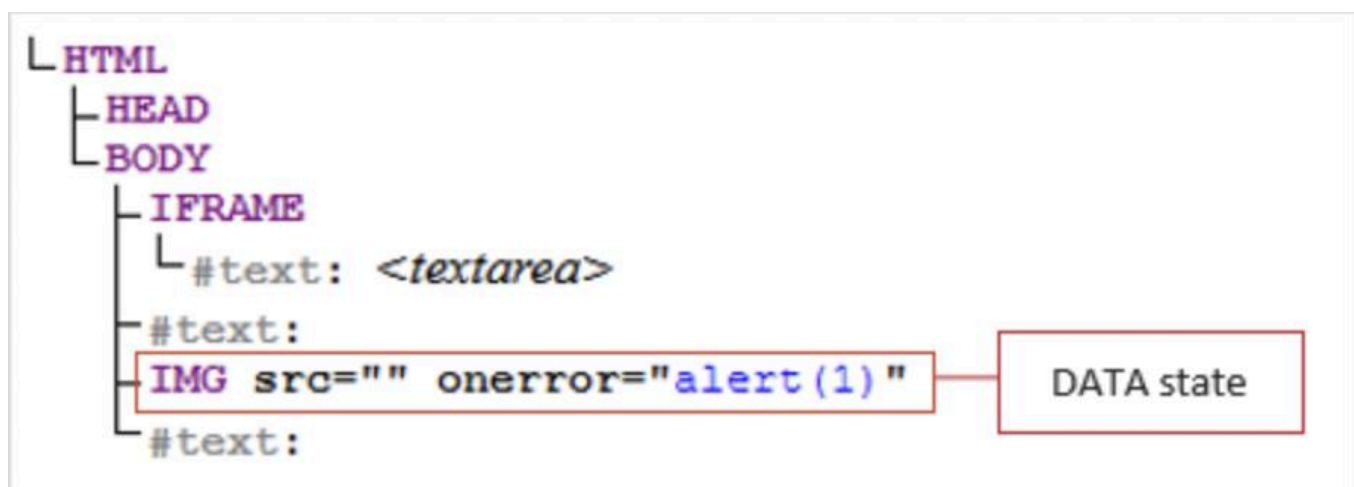
[CVE-2020-12648](#) (XSS in TinyMCE), which was discovered by George Steketee and I, will serve as a test case for how HTML parsing caveats can be leveraged to gain XSS in cases where a sanitizing parser is used. In the TinyMCE advisory, XSS was achieved with the following payload:

```
<iframe><textarea></iframe><img src="" >
```

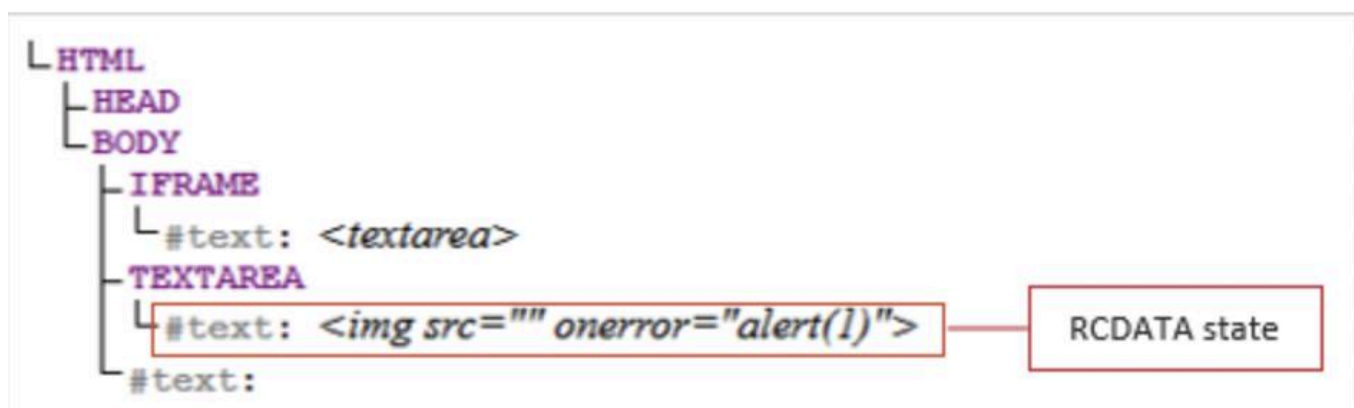
This payload was successful because of an issue in the tokenization and tree construction phases. In particular, when the HTML was reparsed by the lexical parser, it did not properly account for the order of elements before assigning the context state.

The `<iframe>` element caused the context state to switch to RAWTEXT, which meant that the data following the `iframe` was considered not dangerous and did not require sanitization. This context switch ended at the closing tag of `</iframe>`. However, the `<textarea>` element also instructed the parser to switch to the RCDATA context, another form of non-dangerous text data. The context switch to RCDATA was contained within the `iframe` elements when they were processed by the HTML parser. This containment is what the TinyMCE parser failed to realize.

When this was parsed, the TinyMCE parser failed to consider the proper order of operations and context switches. Therefore, the DOM tree construction performed by the final post-sanitization HTML parser looked like this:



The above was a result of TinyMCE's parser viewing the data incorrectly like this:



Note that the `img` element with the active content `onerror` event is within the `text` context in the DOM tree; when lexically parsed, this would register as non-dangerous and not be stripped or output encoded. Since the `textarea` element was contained in the `iframe`, the `img` element was not actually within a `textarea` element. Therefore, the active content (JavaScript) executed and XSS was achieved.

Test Case 2 = Froala XSS

Froala is a What-You-See-Is-What-You-Get (WYSIWYG) HTML text editor and JavaScript library that is similar in functionality to TinyMCE. For a second test case, we will review an XSS vulnerability

that was found as a part of this research ([CVE-2021-28114](#)). In the advisory for this CVE, I detailed how XSS was achieved using the following payload:

[\[image: code for XSS example following payload\]](#)

This payload is functionally the same as the TinyMCE XSS discussed in Test Case 1 of this blog post with one caveat. Entering the MathML namespace to cause parsing confusion (in Froala's instance, restricting a comment within `iframe` elements) was not enough to confuse the Froala parser. However, Froala's parser did not understand `MathMLnamespace` tags and would drop the tags but continue parsing the remaining content. The result was the HTML parser creating nodes with the payload restricted to text data, as shown in the tree below:

[\[image: image\]](#)

However, since Froala's parser omitted the `<math>` element, it would still incorrectly view the `img` element payload as a non-dangerous comment. When the JavaScript payload was processed by the final-stage HTML parser and placed into the DOM, it would do so as follows:

The result was the XSS payload execution. This can be further visualized by examining the post-exploitation source code:

[\[image: Post exploitation source code\]](#)

The Froala parser removed the `<math>` element and added a `-->` to close what it believed was a comment. The final-stage HTML parser viewed the opening comment as contained within `iframe` elements and set the closing comment element added by the Froala parser to the RCDATA state, ignoring it as a valid closing tag. The result was active content execution (XSS).

## Prevention

When implementing applications that allow some user-controlled HTML by design, the key to avoiding these types of bugs is to process the HTML as close to the original parse as possible. While doing so, it is important to account for the order of elements and embed the elements' context. These XSS issues within lexical analysis will arise if there is a variation in how the HTML parser views a node versus how the sanitizing parser views a node. It is also advisable to blacklist MathML and SVG namespace elements when they are not required and completely drop any request containing these (i.e., do not continue to write the data into the DOM).

For organizations that are not creating these types of solutions but rather including them in their applications, a good patch policy will go a long way in preventing exploitation. I recommend checking for the latest versions of these libraries and patching them on a regular and organizationally defined basis.

In addition to security controls at the code/application level, organizations should consider implementing a content security policy (CSP) into the application. A well-defined CSP can block JavaScript injection at a browser-defined level, creating a defense-in-depth security posture. Additionally, the CSP should avoid directives such as `unsafe-eval`, as these can allow user-defined inline JavaScript execution. For more on CSP, please refer to this informative article.

## Conclusion

---



Even when input is lexically analyzed, XSS may still be possible by exploiting caveats in how HTML is parsed and reparsed by whatever sanitization library is in use. When testing for this type of XSS, I recommend fuzzing inputs with various namespace and context switching elements, recording any interesting results, and working off those results.

## Resources

---

- *\*Securitem - DOM Purify Bypass* - [\\*https://research.securitem.com...](https://research.securitem.com...)
- **Bishop Fox – TinyMCE v5.2.1 Advisory**
- *\*Mozilla – Content Security Policy* - [\\*https://developer.mozilla.org/...](https://developer.mozilla.org/...)

---

Archived from <https://bishopfox.com/blog/lexss-bypassing-lexical-parsing-security-controls>. Rendered offline from the local Markdown copy.